거버넌스 서비스 문서

소개

거버넌스 서비스는 Oasis Core 네트워크의 탈중앙화된 의사결정 시스템을 제공하는 핵심 프로세스입니다. 이 서비스를 통해 네트워크 참여자들은 네트워크 업그레이드, 매개변수 변경, 기타 중요한 네트워크 정책에 대한 제안을 생성하고 투표할 수 있습니다.

주요 특징

- **탈중앙화된 제안 시스템**: 누구나 최소 예치금을 통해 제안을 생성할 수 있습니다
- **가중 투표 메커니즘**: 스테이킹된 토큰 양에 비례한 투표권을 제공합니다
- **자동 실행**: 통과된 제안은 자동으로 실행됩니다
- **다양한 제안 유형**: 업그레이드, 매개변수 변경, 업그레이드 취소 제안을 지원합니다

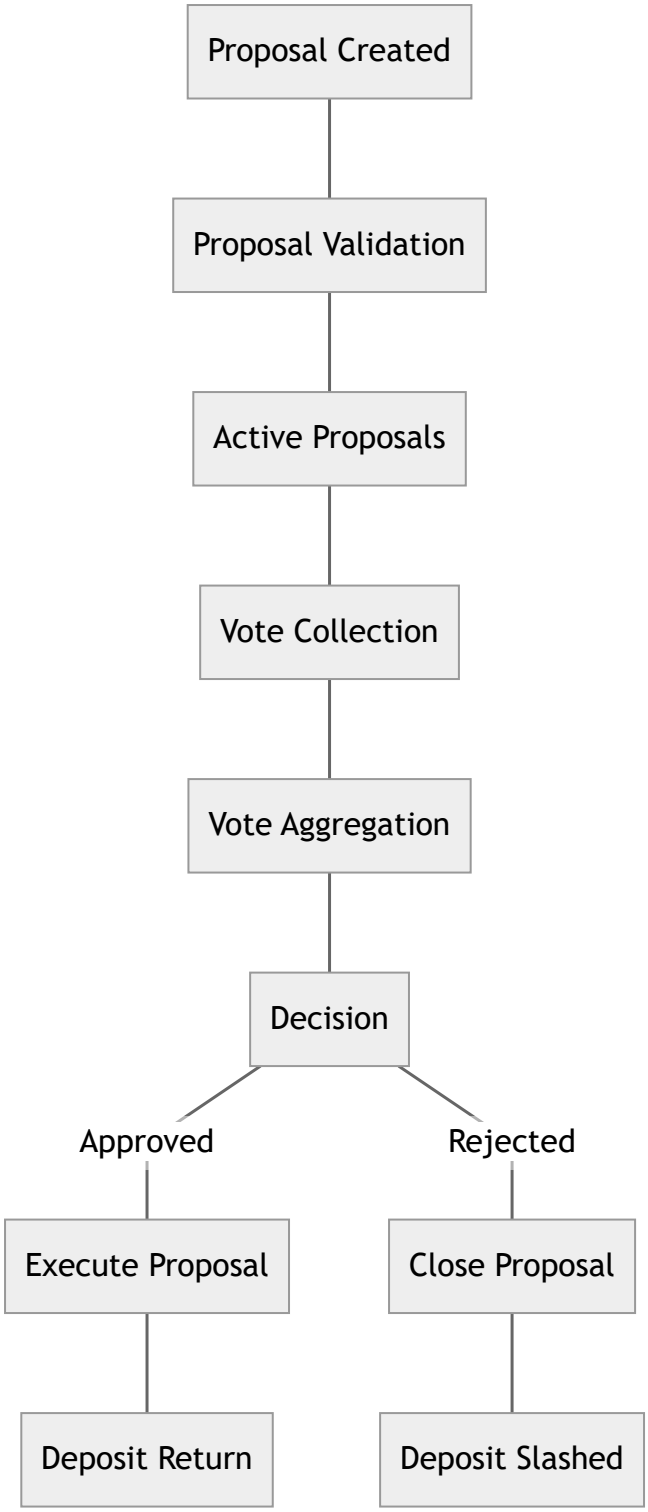
관련 프로세스

- **스테이킹 서비스**: 투표권 계산과 예치금 관리를 위한 스테이킹 정보 제공
- **스케줄러 서비스**: 현재 밸리데이터 세트 정보 제공으로 투표 자격 확인
- **레지스트리 서비스**: 엔티티 정보를 통한 투표자 자격 검증
- **업그레이드 서비스**: 네트워크 업그레이드 제안의 실행과 관리

아키텍처

전체 구조

거버넌스 서비스는 제안 생성, 투표 수집, 결과 집계, 제안 실행의 전체 생명주기를 관리합니다.



주요 컴포넌트

제안 관리자 (Proposal Manager)

역할: 제안의 생성, 검증, 상태 관리를 담당합니다

주요 기능:

- 제안 내용 검증 및 저장

- 제안 상태 추적 (활성, 통과, 거부, 실패)
- 제안 생명주기 관리

투표 집계기 (Vote Aggregator)

역할: 투표를 수집하고 가중치를 계산하여 최종 결과를 도출합니다

주요 기능:

- 투표자 자격 검증
- 스테이킹 기반 가중치 계산
- 위임 투표 처리
- 최종 투표 결과 집계

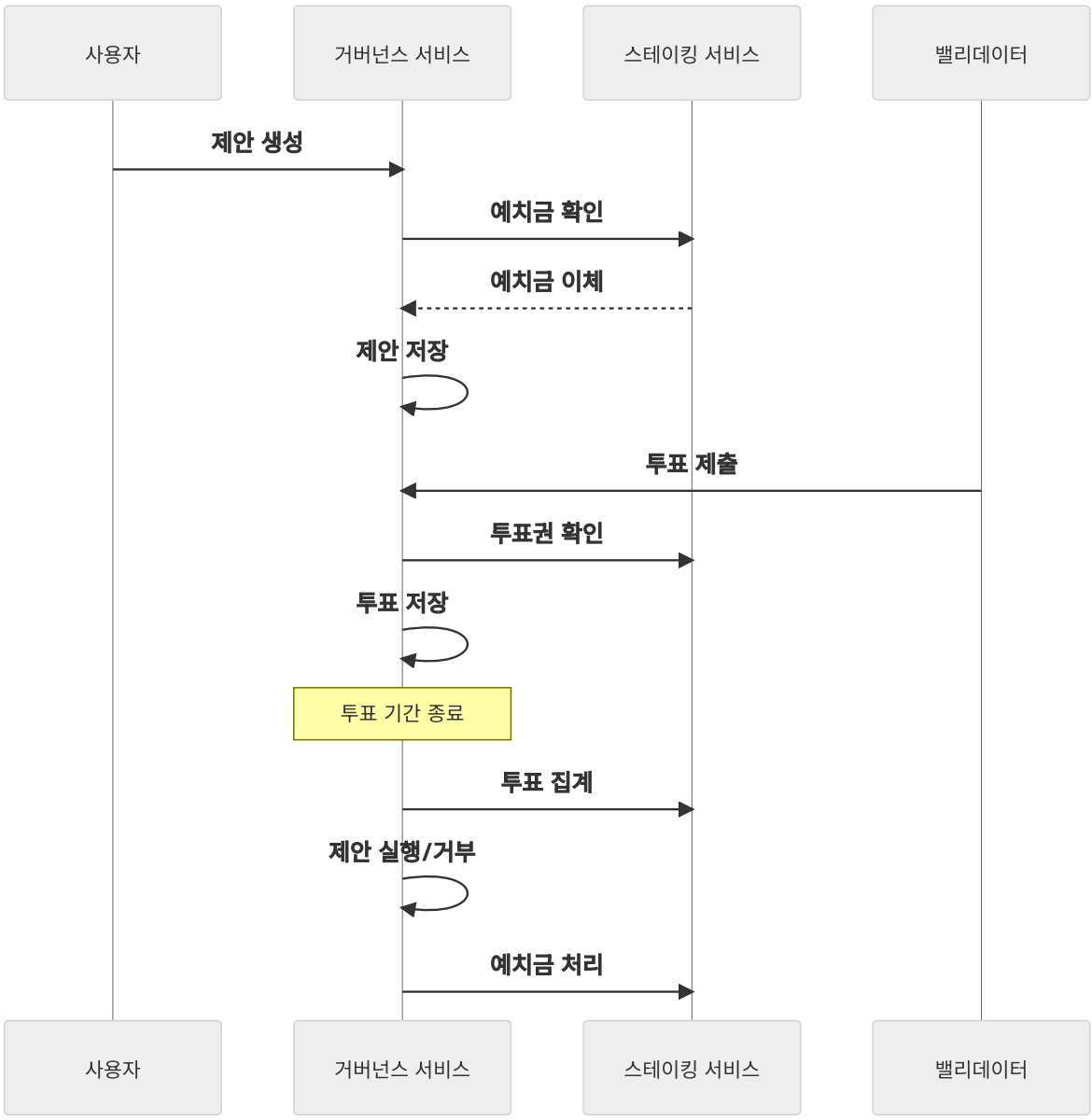
제안 실행기 (Proposal Executor)

역할: 통과된 제안의 내용을 실제로 실행합니다

주요 기능:

- 업그레이드 제안 실행
- 매개변수 변경 적용
- 업그레이드 취소 처리

데이터 흐름



API 및 인터페이스

주요 인터페이스

Backend 인터페이스

```go
type Backend interface {
// ActiveProposals returns a list of all proposals that have not yet closed.
ActiveProposals(ctx context.Context, height  int64) ([]*Proposal, error)
// Proposals returns a list of all proposals.
Proposals(ctx context.Context, height  int64) ([]*Proposal, error)
// Proposal looks up a specific proposal.
Proposal(ctx context.Context, query *ProposalQuery) (*Proposal, error)
// Votes looks up votes for a specific proposal.
Votes(ctx context.Context, query *ProposalQuery) ([]*VoteEntry, error)
// PendingUpgrades returns a list of all pending upgrades.
PendingUpgrades(ctx context.Context, height  int64) ([]*upgrade.Descriptor, erro
// ConsensusParameters returns the governance consensus parameters.
ConsensusParameters(ctx context.Context, height  int64) (*ConsensusParameters, e
// GetEvents returns the events at specified block height.
GetEvents(ctx context.Context, height  int64) ([]*Event, error)
// WatchEvents returns a channel that produces a stream of Events.
WatchEvents(ctx context.Context) (<-chan *Event, pubsub.ClosableSubscription, er
}
```

메서드 설명:

- `ActiveProposals` : 현재 활성 상태인 모든 제안 목록을 반환합니다
- `height` : 조회할 블록 높이

- 반환값: 활성 제안 목록
- 에러: 상태 조회 실패 시
- `Proposal` : 특정 제안의 상세 정보를 조회합니다
- `query` : 제안 ID와 블록 높이를 포함한 쿼리
- 반환값: 제안 상세 정보
- 에러: 제안이 존재하지 않거나 조회 실패 시
- `Votes` : 특정 제안에 대한 모든 투표 정보를 조회합니다
- `query` : 제안 ID와 블록 높이를 포함한 쿼리
- 반환값: 투표 엔트리 목록
- 에러: 제안이 존재하지 않거나 조회 실패 시

트랜잭션 메서드

제안 생성

```
// MethodSubmitProposal submits a new consensus layer governance proposal.

MethodSubmitProposal = transaction.NewMethodName(ModuleName, "SubmitProposal", P
```

투표 제출

```
// MethodCastVote casts a vote for a consensus layer governance proposal.

MethodCastVote = transaction.NewMethodName(ModuleName, "CastVote", ProposalVote{
```

gRPC API

서비스 정의

거버넌스 서비스는 다음과 같은 gRPC 메서드를 제공합니다:

- `ActiveProposals` : 활성 제안 목록 조회
- `Proposals` : 모든 제안 목록 조회
- `Proposal` : 특정 제안 조회
- `Votes` : 제안별 투표 목록 조회
- `PendingUpgrades` : 대기 중인 업그레이드 목록 조회
- `ConsensusParameters` : 거버넌스 합의 매개변수 조회
- `GetEvents` : 특정 높이의 이벤트 조회
- `WatchEvents` : 실시간 이벤트 스트림 구독

제안 생성과 검증 프로세스

제안 유형

거버넌스 시스템은 세 가지 주요 제안 유형을 지원합니다:

1. 업그레이드 제안 (UpgradeProposal)

네트워크 업그레이드를 위한 제안입니다.

```
type UpgradeProposal struct {
upgrade.Descriptor
}
```

검증 조건:

- 업그레이드 에포크가 현재 에포크보다 `UpgradeMinEpochDiff` 이상 미래여야 함

- 동일한 시기에 다른 업그레이드가 예정되어 있지 않아야 함

2. 업그레이드 취소 제안 (CancelUpgradeProposal)

기존 업그레이드 제안을 취소하는 제안입니다.

```
type CancelUpgradeProposal struct {
ProposalID  uint64  `json:"proposal_id"`
}
```

검증 조건:

- 취소하려는 제안이 실제로 존재해야 함
- 업그레이드 에포크가 `UpgradeCancelMinEpochDiff` 이상 미래여야 함

3. 매개변수 변경 제안 (ChangeParametersProposal)

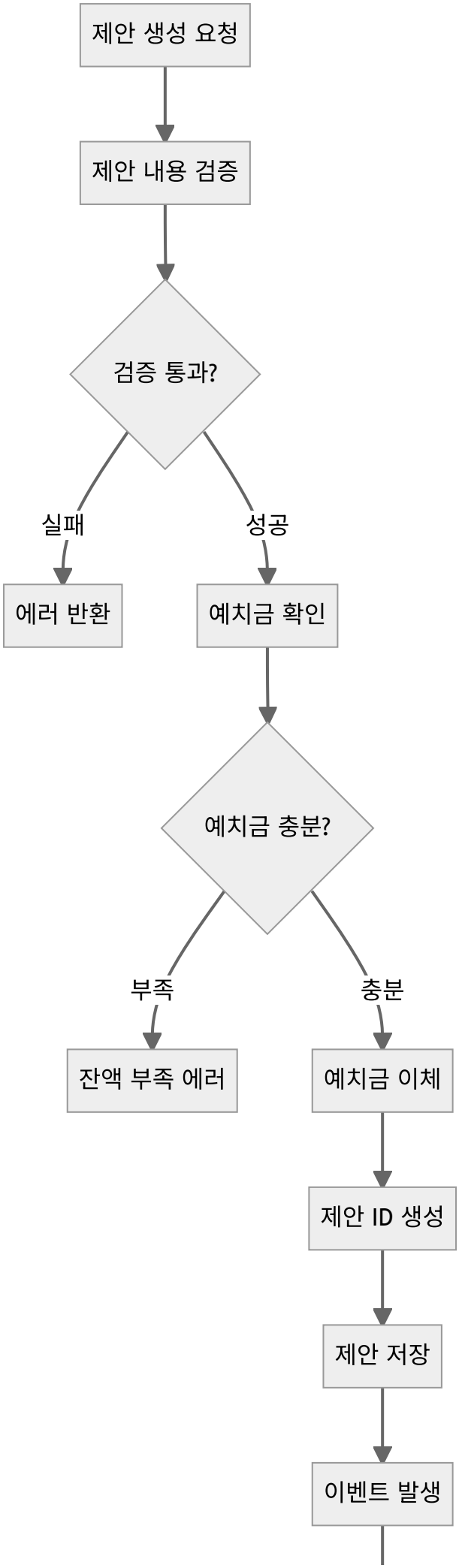
네트워크 매개변수를 변경하는 제안입니다.

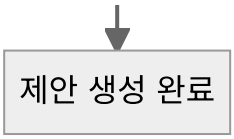
```
type ChangeParametersProposal struct {
Module  string  `json:"module"`
Changes cbor.RawMessage `json:"changes"`
}
```

검증 조건:

- 모듈 이름이 유효해야 함
- 변경 내용이 비어있지 않아야 함
- 해당 모듈에서 변경을 지원해야 함

제안 생성 프로세스


제안 생성 요청
제안 내용 검증
검증 통과?
실패
성공
에러 반환
예치금 확인
예치금 충분?
부족
충분
잔액 부족 에러
예치금 이체
제안 ID 생성
제안 저장
이벤트 발생



제안 검증 규칙

1. **기본 검증**:

- 제안 메타데이터 형식 확인
- 제안 내용 유효성 검사
- 하나의 제안 유형만 포함되어야 함

2. **예치금 검증**:

- 제안자가 최소 예치금 이상을 보유해야 함
- 예치금은 거버넌스 예치 계정으로 이체됨

3. **타이밍 검증**:

- 업그레이드 제안: 충분한 미래 에포크 지정
- 취소 제안: 취소 가능한 시점 내에서만 허용

투표 메커니즘과 가중치 계산

투표 자격

투표 자격은 다음 조건 중 하나를 만족해야 합니다:

1. **밸리데이터**: 현재 밸리데이터 세트에 포함된 엔티티
2. **위임자**: 현재 밸리데이터에게 토큰을 위임한 계정

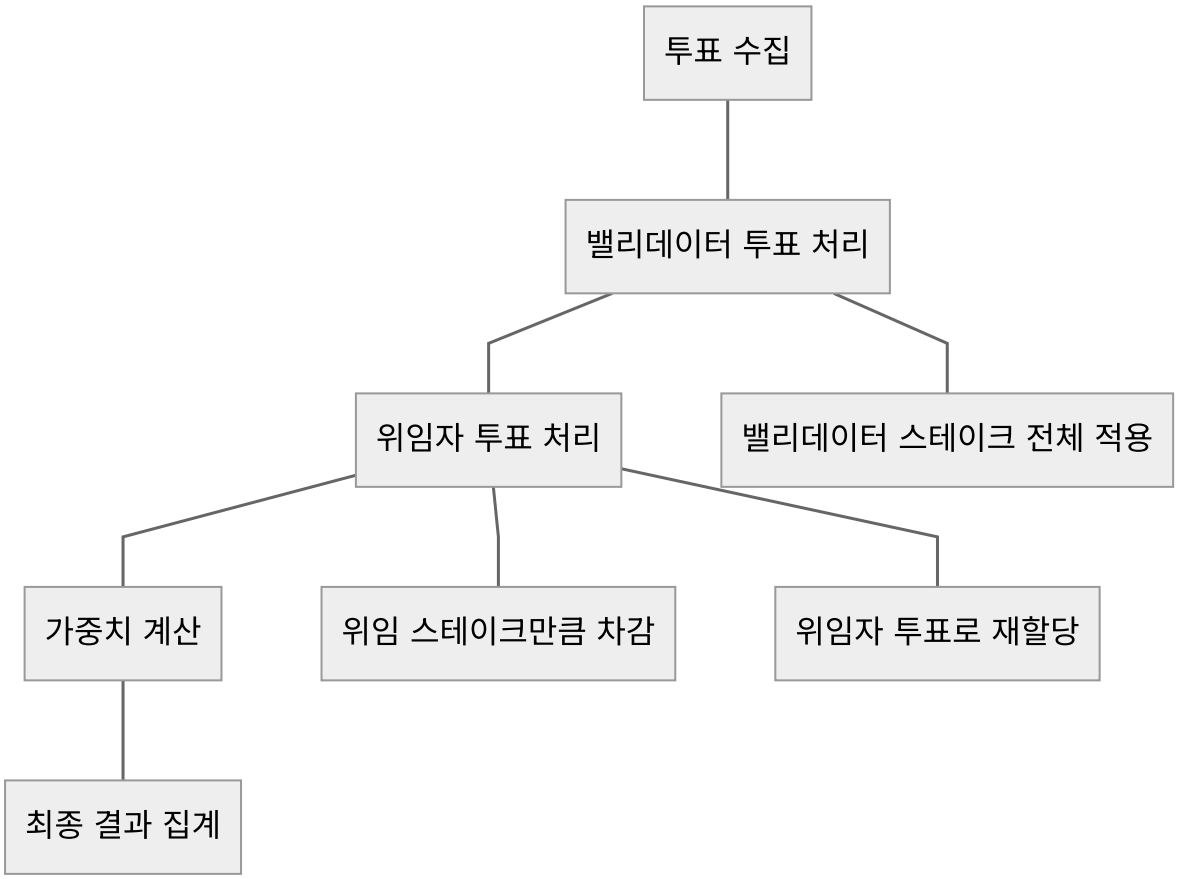
투표 유형

```
type Vote uint8
```

```
const (
VoteYes Vote = 1  // 찬성
VoteNo Vote = 2  // 반대
VoteAbstain Vote = 3  // 기권
)
```

가중치 계산 방식

투표 가중치는 스테이킹된 토큰 양에 비례하여 계산됩니다:



1. 밸리데이터 투표

- 밸리데이터의 전체 스테이크(자체 + 위임받은 스테이크)가 투표 가중치로 적용됩니다
- 위임자가 별도로 투표하지 않은 경우, 밸리데이터의 투표가 모든 위임 스테이크에 적용됩니다

2. 위임자 투표

- 위임자가 직접 투표하는 경우, 해당 위임 스테이크는 밸리데이터 투표에서 차감됩니다
- 위임자의 투표에 해당 위임 스테이크가 가중치로 적용됩니다

3. 가중치 계산 공식

```
최종_투표_가중치 = Σ(밸리데이터_스테이크 - 위임자_직접_투표_스테이크) + Σ(위임자_직접_투표
```

투표 집계 프로세스

```go
// 투표 집계 의사코드
func (app *Application) closeProposal(
ctx *api.Context,
proposal *governance.Proposal,
totalVotingStake quantity.Quantity,
validatorEntitiesPool  map[stakingAPI.Address]*stakingAPI.SharePool,
) error {
// 1. 밸리데이터 투표 수집
validatorVotes := collectValidatorVotes(proposal.ID)
// 2. 위임자 투표 수집 및 가중치 재계산
delegatorVotes := collectDelegatorVotes(proposal.ID)
adjustValidatorWeights(validatorVotes, delegatorVotes)
// 3. 최종 결과 계산
results := calculateFinalResults(validatorVotes, delegatorVotes)
```

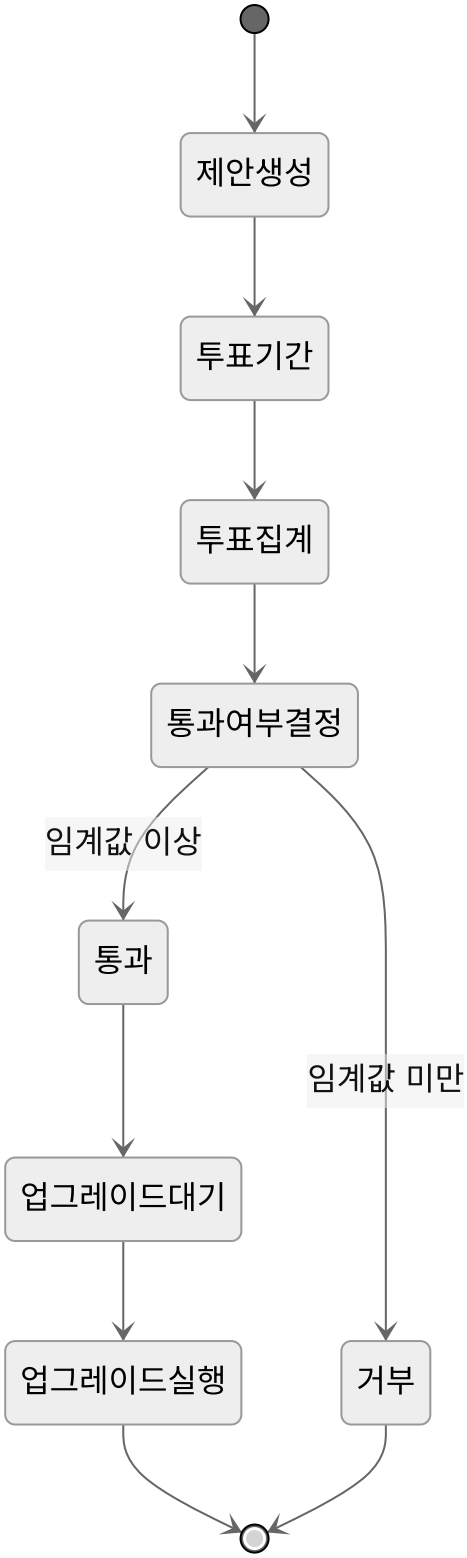
```
// 4. 통과 여부 결정

return  proposal.CloseProposal(totalVotingStake, stakeThreshold)

}
```

네트워크 업그레이드 절차

업그레이드 제안 생명주기



업그레이드 실행 프로세스

1. 제안 통과 후 처리

```
func (app *Application) executeProposal(
ctx *api.Context,
```

```
state *governanceState.MutableState,
proposal *governance.Proposal,
) error {
switch {
case  proposal.Content.Upgrade != nil:
// 업그레이드 제안 실행
err := state.SetPendingUpgrade(ctx, proposal.ID, &proposal.Content.Upgrade.Descr
if  err != nil {
return  fmt.Errorf("failed to set pending upgrade: %w", err)
}
// 로컬 업그레이더에 업그레이드 정보 전달
if  upgrader := ctx.AppState().Upgrader();  upgrader != nil {
err = upgrader.SubmitDescriptor(&proposal.Content.Upgrade.Descriptor)
}
}
return  nil
}
```

2. 업그레이드 검증 및 실행

각 에포크 시작 시 대기 중인 업그레이드를 확인합니다:

```
func (app *Application) BeginBlock(ctx *api.Context) error {
epochChanged, epoch := app.state.EpochChanged(ctx)
```

```
if !epochChanged {
return nil
}
// 현재 에포크에 예정된 업그레이드 확인
pendingUpgrades, err := state.PendingUpgrades(ctx)
if err != nil {
return err
}
for _, upgrade := range pendingUpgrades {
if upgrade.Epoch == epoch {
// 업그레이드 실행 또는 검증
return app.handleUpgrade(ctx, upgrade)
}
}
return nil
}
```

3. 업그레이드 취소

업그레이드 취소 제안이 통과되면 대기 중인 업그레이드를 제거합니다:

```
case proposal.Content.CancelUpgrade != nil:
// 취소할 업그레이드 제안 조회
cancelingProposal, err := state.Proposal(ctx, proposal.Content.CancelUpgrade.Pro
// 대기 중인 업그레이드 제거
```

```
err = state.RemovePendingUpgrade(ctx,

cancelingProposal.Content.Upgrade.Epoch,

cancelingProposal.ID)

// 로컬 업그레이더에 취소 알림

if  upgrader := ctx.AppState().Upgrader();  upgrader != nil {

err = upgrader.CancelUpgrade(&upgradeProposal.Descriptor)

}
```

업그레이드 안전장치

1. **최소 에포크 차이**: 업그레이드는 현재 에포크로부터 최소 `UpgradeMinEpochDiff` 에포크 후에만 실행 가능
2. **중복 업그레이드 방지**: 동일한 시기에 여러 업그레이드가 예정되지 않도록 제한
3. **취소 기한**: 업그레이드 취소는 `UpgradeCancelMinEpochDiff` 에포크 전까지만 가능

거버넌스 API와 예제

제안 생성 예제

```
package main


import (

"context"

"fmt"

"github.com/oasisprotocol/oasis-core/go/governance/api"
```

```go
"github.com/oasisprotocol/oasis-core/go/consensus/api/transaction"
)

func  submitUpgradeProposal() error {
// 업그레이드 제안 내용 생성
proposalContent := &api.ProposalContent{
Metadata: &api.ProposalMetadata{
Title: "네트워크 업그레이드 v2.0",
Description: "새로운 기능과 보안 개선사항을 포함한 메이저 업그레이드",
},
Upgrade: &api.UpgradeProposal{
Descriptor: upgrade.Descriptor{
Versioned: cbor.NewVersioned(upgrade.LatestDescriptorVersion),
Handler: "upgrade-v2.0",
Target: version.Versions,
Epoch: 1000, // 업그레이드 실행 에포크
},
},
}
// 트랜잭션 생성
tx := api.NewSubmitProposalTx(
nonce, // 논스
&transaction.Fee{}, // 수수료
```

```
proposalContent, // 제안 내용
)
// 트랜잭션 서명 및 제출
return  submitTransaction(tx)
}
```

투표 제출 예제

```
func  castVote(proposalID  uint64, vote api.Vote) error {
// 투표 내용 생성
proposalVote := &api.ProposalVote{
ID: proposalID,
Vote: vote,
}
// 트랜잭션 생성
tx := api.NewCastVoteTx(
nonce, // 논스
&transaction.Fee{}, // 수수료
proposalVote, // 투표 내용
)
// 트랜잭션 서명 및 제출
return  submitTransaction(tx)
}
```

```go
// 사용 예시
func main() {
// 제안 ID 1번에 찬성 투표
err := castVote(1, api.VoteYes)
if err != nil {
fmt.Printf("투표 실패: %v\n", err)
return
}
fmt.Println("투표가 성공적으로 제출되었습니다")
}
```

제안 조회 예제

```go
func queryProposals(client api.Backend) error {
ctx := context.Background()
// 활성 제안 목록 조회
activeProposals, err := client.ActiveProposals(ctx, consensusAPI.HeightLatest)
if err != nil {
return fmt.Errorf("활성 제안 조회 실패: %w", err)
}
fmt.Printf("활성 제안 수: %d\n", len(activeProposals))
for _, proposal := range activeProposals {
```

```
fmt.Printf("제안 ID: %d\n", proposal.ID)

fmt.Printf("제안자: %s\n", proposal.Submitter)

fmt.Printf("상태: %s\n", proposal.State)

fmt.Printf("마감 에포크: %d\n", proposal.ClosesAt)

if  proposal.Content.Metadata != nil {

fmt.Printf("제목: %s\n", proposal.Content.Metadata.Title)

}

// 투표 결과 조회

votes, err := client.Votes(ctx, &api.ProposalQuery{

Height: consensusAPI.HeightLatest,

ProposalID: proposal.ID,

})

if  err == nil {

fmt.Printf("총 투표 수: %d\n", len(votes))

}

fmt.Println("---")

}

return  nil

}
```

CLI 사용 예제

```
# 활성 제안 목록 조회
```

```
oasis-node governance list-proposals --active

# 특정 제안 상세 조회
oasis-node governance show-proposal --proposal-id 1

# 제안에 투표하기
oasis-node governance cast-vote \
--proposal-id 1 \
--vote yes \
--account my-account

# 대기 중인 업그레이드 조회
oasis-node governance pending-upgrades

# 거버넌스 매개변수 조회
oasis-node governance parameters
```

매개변수 변경 제안 예제

```
func submitParameterChangeProposal() error {
// 스테이킹 매개변수 변경 예시
changes := map[string]interface{}{
"min_delegation": "1000000000", // 최소 위임 금액 변경
```

```
"reward_factor_epoch_signed": "1.0",
}
changesBytes, err := cbor.Marshal(changes)
if err != nil {
return err
}
proposalContent := &api.ProposalContent{
Metadata: &api.ProposalMetadata{
Title: "스테이킹 매개변수 조정",
Description: "최소 위임 금액을 1 ROSE로 조정하여 네트워크 참여 장벽을 낮춤",
},
ChangeParameters: &api.ChangeParametersProposal{
Module: "staking",
Changes: changesBytes,
},
}
tx := api.NewSubmitProposalTx(nonce, &transaction.Fee{}, proposalContent)
return submitTransaction(tx)
}
```

설정 및 구성

거버넌스 합의 매개변수

```go
type ConsensusParameters struct {
// 가스 비용 설정
GasCosts transaction.Costs `json:"gas_costs,omitempty"`
// 최소 제안 예치금 (base units)
MinProposalDeposit quantity.Quantity `json:"min_proposal_deposit,omitempty"`
// 투표 기간 (에포크 수)
VotingPeriod beacon.EpochTime `json:"voting_period,omitempty"`
// 통과 임계값 (퍼센트, 최소 67)
StakeThreshold  uint8  `json:"stake_threshold,omitempty"`
// 업그레이드 최소 에포크 차이
UpgradeMinEpochDiff beacon.EpochTime `json:"upgrade_min_epoch_diff,omitempty"`
// 업그레이드 취소 최소 에포크 차이
UpgradeCancelMinEpochDiff beacon.EpochTime `json:"upgrade_cancel_min_epoch_diff,
// 매개변수 변경 제안 허용 여부
EnableChangeParametersProposal  bool  `json:"enable_change_parameters_proposal,o
// 엔티티 없이 투표 허용 여부
AllowVoteWithoutEntity  bool  `json:"allow_vote_without_entity,omitempty"`
// 제안 메타데이터 허용 여부
AllowProposalMetadata  bool  `json:"allow_proposal_metadata,omitempty"`
}
```

주요 설정 옵션

| 매개변수 | 기본값 | 설명 |

|----------|--------|------|

| `MinProposalDeposit` | 네트워크별 상이 | 제안 생성 시 필요한 최소 예치금 |

| `VotingPeriod` | 168 에포크 | 제안 투표 기간 (약 1주일) |

| `StakeThreshold` | 67% | 제안 통과를 위한 최소 찬성 비율 |

| `UpgradeMinEpochDiff` | 336 에포크 | 업그레이드 최소 예고 기간 (약 2주일) |

| `EnableChangeParametersProposal` | false | 매개변수 변경 제안 활성화 여부 |

가스 비용 설정

```
var DefaultGasCosts = transaction.Costs{
GasOpSubmitProposal: 1000, // 제안 생성 가스 비용
GasOpCastVote: 1000, // 투표 제출 가스 비용
}
```

모니터링 및 이벤트

거버넌스 이벤트

거버넌스 서비스는 다음과 같은 이벤트를 발생시킵니다:

1. 제안 생성 이벤트

```
type ProposalSubmittedEvent struct {
ID uint64 `json:"id"` // 제안 ID
Submitter staking.Address `json:"submitter"` // 제안자 주소
```

```
}
```

2. 투표 이벤트

```
type VoteEvent struct {
ID  uint64  `json:"id"`  // 제안 ID
Submitter staking.Address `json:"submitter"`  // 투표자 주소
Vote Vote `json:"vote"`  // 투표 내용
}
```

3. 제안 완료 이벤트

```
type ProposalFinalizedEvent struct {
ID  uint64  `json:"id"`  // 제안 ID
State ProposalState `json:"state"`  // 최종 상태
}
```

4. 제안 실행 이벤트

```
type ProposalExecutedEvent struct {
ID  uint64  `json:"id"`  // 제안 ID
}
```

이벤트 구독 예제

```go
func watchGovernanceEvents(client api.Backend) error {
ctx := context.Background()
// 이벤트 스트림 구독
eventCh, sub, err := client.WatchEvents(ctx)
if err != nil {
return fmt.Errorf("이벤트 구독 실패: %w", err)
}
defer sub.Close()
for {
select {
case event := <-eventCh:
switch {
case event.ProposalSubmitted != nil:
fmt.Printf("새 제안 생성: ID=%d, 제안자=%s\n",
event.ProposalSubmitted.ID,
event.ProposalSubmitted.Submitter)
case event.Vote != nil:
fmt.Printf("투표 제출: 제안ID=%d, 투표자=%s, 투표=%s\n",
event.Vote.ID,
event.Vote.Submitter,
event.Vote.Vote)
case event.ProposalFinalized != nil:
```

```
fmt.Printf("제안 완료: ID=%d, 상태=%s\n",
event.ProposalFinalized.ID,
event.ProposalFinalized.State)
case  event.ProposalExecuted != nil:
fmt.Printf("제안 실행: ID=%d\n", event.ProposalExecuted.ID)
}
case <-ctx.Done():
return  ctx.Err()
}
}
}
```

문제 해결

일반적인 문제

문제 1: 제안 생성 실패 - 예치금 부족

증상: `ErrInsufficientBalance` 에러와 함께 제안 생성이 실패합니다

원인: 제안자 계정의 잔액이 최소 예치금보다 적습니다

해결 방법:

1. 현재 거버넌스 매개변수에서 최소 예치금 확인
2. 계정 잔액 확인 및 충분한 토큰 확보
3. 제안 재생성

```
# 최소 예치금 확인
oasis-node governance parameters | grep min_proposal_deposit

# 계정 잔액 확인
oasis-node staking account info --account.address <address>
```

문제 2: 투표 실패 - 자격 없음

증상: `ErrNotEligible` 에러와 함께 투표가 거부됩니다

원인: 투표자가 현재 밸리데이터가 아니고 밸리데이터에게 위임도 하지 않았습니다

해결 방법:

1. 현재 밸리데이터 세트 확인
2. 밸리데이터에게 토큰 위임
3. 또는 밸리데이터 노드 운영

```
# 현재 밸리데이터 확인
oasis-node scheduler validators

# 위임 상태 확인
oasis-node staking account info --account.address <address>
```

문제 3: 업그레이드 제안 실패 - 시기 부적절

증상: `ErrUpgradeTooSoon` 에러와 함께 업그레이드 제안이 거부됩니다

원인: 업그레이드 에포크가 현재 에포크로부터 충분히 미래가 아닙니다

해결 방법:

1. 현재 에포크 확인
2. 최소 에포크 차이 매개변수 확인
3. 적절한 미래 에포크로 업그레이드 제안 수정

```
# 현재 에포크 확인
oasis-node control status

# 거버넌스 매개변수 확인
oasis-node governance parameters | grep upgrade_min_epoch_diff
```

로그 분석

```
# 거버넌스 관련 로그 확인
grep "governance" /var/log/oasis/node.log

# 제안 관련 에러 로그
grep "proposal.*error\|failed.*proposal" /var/log/oasis/node.log

# 투표 관련 로그
grep "vote\|voting" /var/log/oasis/node.log
```

참조

관련 문서

- 스테이킹 서비스: 투표권 계산의 기반이 되는 스테이킹 시스템
- 스케줄러 서비스: 밸리데이터 세트 관리
- 레지스트리 서비스: 엔티티 등록 및 관리

소스 코드

- 거버넌스 API
- 거버넌스 구현
- 거버넌스 상태 관리

외부 리소스

- Oasis 거버넌스 가이드
- 네트워크 업그레이드 절차
- 거버넌스 제안 예시

마지막 업데이트: 2024년 12월

작성자: Oasis Core 문서화 팀

검토자: 거버넌스 개발팀